

SERVICIO NACIONAL DE APRENDIZAJE – SENA

Regional Boyacá · Centro Industrial de Mantenimiento y Manufactura – CIMM Tecnólogo
en Análisis y Desarrollo de Software – ADSO

TALLER PRÁCTICO

Internacionalización (i18n) + Docker + OTP

Aplicado al Sistema de Gestión de Vacunación

Java Servlets · JSP · JSTL · MySQL · Docker · Jakarta Mail

Español · English · Français

Proyecto base:	captcha.war — Sistema de Gestión de Vacunación
Tecnología:	Java EE · Servlets · JSP · JSTL · MySQL · Tomcat 9
Competencia:	225501003 – Desarrollar e integrar componentes y servicios
Hosting gratuitos:	Render · Koyeb · Google Cloud Run · Oracle Cloud · Back4App
Duración estimada:	14 horas — 3 sesiones de laboratorio
Versión:	1.0 – 2026

Repositorio de GitHub:

<https://github.com/juliandreyes23/SaludBoyac-.git>

Video:

<https://drive.google.com/drive/folders/1qJOgPgMPjkkP4jRA1bEtSI3cYzJSmv6p?usp=sharing>

1. Introducción y Contexto

El instructor entrega el código fuente completo del Sistema de Gestión de Vacunación (captcha.war) como material de referencia. Este sistema Java EE con Servlets, JSP/JSTL y MySQL ya implementa i18n en 4 idiomas (es, en, fr, it), CAPTCHA gráfico, exportación a PDF con iText, exportación a Excel con Apache POI, autenticación por roles y consulta pública.

¿Para qué sirve el código de vacunación en este taller?

El sistema de vacunación es tu GUÍA DE ARQUITECTURA, no el sistema a modificar.

Estudia cómo está construido para entender:

- Cómo funciona LocaleFilter.java + archivos .properties + <fmt:message> (i18n)
- Cómo está estructurado el LoginServlet, AuthFilter y los Servlets CRUD
- Cómo se genera el PDF, cómo funciona el DAO y cómo se conecta a MySQL
- Cómo está organizado el pom.xml con sus dependencias

Luego aplicarás EXACTAMENTE los mismos patrones en una aplicación NUEVA,

diferente en dominio y contexto, pero idéntica en arquitectura.

No se trata de copiar — se trata de entender y replicar con criterio.

CAPTCHA vs OTP — ¿por qué OTP reemplaza al CAPTCHA en el login?

El CAPTCHA gráfico protege contra bots automatizados que prueban credenciales.

El OTP (One-Time Password) por correo también cumple esa función (un bot no tiene acceso al correo del usuario), pero además agrega verificación de identidad real:

demuestra que quien inicia sesión tiene acceso al correo registrado en la BD.

Por tanto: en el formulario de LOGIN de tu nueva aplicación NO usarás CAPTCHA.

Solo usuario + contraseña → si son válidos → enviar OTP al correo → verificar código.

Excepción: el módulo de CONSULTA PÚBLICA mantiene el CAPTCHA porque allí

no hay cuenta de usuario ni correo al que enviar un OTP.

El taller tiene dos partes. La Parte A (secciones 2 a 8) estudia el código de vacunación para entender cada componente: la i18n, el OTP, Docker y los 5 hosting. La Parte B (sección 9) es el proyecto final: construir SaludBoyacá desde cero aplicando todo lo aprendido.

2. Parte A — Análisis del Sistema de Vacunación (Código de Referencia)

Antes de construir tu aplicación, debes entender a fondo el código del sistema de vacunación que el instructor te entregó. Esta sección te guía en ese análisis. No modifiques nada — solo lee, entiende y toma notas en tu bitácora.

¿Qué debes analizar y documentar en tu bitácora?

Por cada componente de la tabla siguiente, responde en tu bitácora:

- ¿Qué hace exactamente esta clase/archivo?
- ¿Qué patrón de diseño aplica?
- ¿Cómo lo voy a replicar en mi aplicación SaludBoyacá?
- ¿Qué adaptaciones necesito hacer para mi contexto?

Clase / Archivo	Paquete / Ruta	Qué aprender de él para tu proyecto	¿Qué hace exactamente?	¿Qué patrón aplica?	¿Cómo lo voy a replicar en SaludBoyacá?
Conexion.java	model/	Patrón de conexión MySQL con getConnection(). En tu proyecto: adaptarlo a variables de entorno (DB_URL, DB_USER, DB_PASS).	Gestiona el enlace físico entre la aplicación Java y la base de datos MySQL	Singleton / Factory Method	Creando una clase centralizada para el canal de comunicación con la base de datos
AuthFilter.java	util/	Cómo proteger rutas con @WebFilter. En tu proyecto: añadir la condición otpVerificado=true.	Intercepta las solicitudes HTTP para verificar si el usuario tiene una sesión activa antes de entrar a rutas privadas	Filter(Interceptor)	Lo usaré para bloquear el acceso al panel médico si el usuario no está logueado
LocaleFilter.java	util/	Cómo detectar ?lang=XX y guardar en sesión. Reutilizable casi sin cambios en tu proyecto.	Detecta el parámetro de idioma en la URL y configura la región del sistema (i18n)	Filter(Interceptor)	Lo copiaré casi tal cual para manejar el cambio de idioma dinámico en el sistema
CaptchaGenerator.java	util/	Cómo genera y valida el CAPTCHA. En tu proyecto: solo para la consulta pública.	Genera una imagen aleatoria con un código y lo guarda en sesión para validar que el usuario es humano	Utility Class / Image Stream	Lo implementaré únicamente para el formulario de consulta pública y consultas médicas
LoginServlet.java	controller/	Flujo doGet/doPost. En tu proyecto: eliminar el CAPTCHA y en su lugar generar OTP.	Procesa las credenciales de acceso, valida el usuario y redirige según el éxito del ingreso	Controller (MVC)	Será la base de mi autenticación, manejando el doPost para recibir usuario y clave.
DashboardServlet.java	controller/	Cómo cargar estadísticas según rol. Replicar la lógica de rol para tu dashboard de citas.	Recopila datos del modelo para mostrar estadísticas y resúmenes en la página principal	Controller (MVC)	Lo usaré para que el médico vea las citas que tiene hoy el paciente y las próximas citas.
PacienteServlet.java	controller/	Patrón CRUD completo con switch de acción. Idéntico patrón para CitaServlet y	Centraliza todas las operaciones de Crear, Leer, Actualizar y Borrar pacientes	Controller (CRUD con Switch)	Usaré este mismo esquema de "acciones" para gestionar tanto pacientes como las citas médicas

		PacienteServlet en tu proyecto.			
PDFGenerator.java	util/	Cómo genera PDFs con iText. En tu proyecto: adaptarlo para generar comprobantes de cita.	Toma datos del sistema y utiliza la librería iText para maquetar y descargar un archivo PDF	Strategy / Utility Class	Lo implementaré para que el p descargue su comprobante de asignación de cita
messages.properties (x4)	resources/	Estructura de claves i18n. Tu proyecto necesita 3 archivos (es, en, it) con claves diferentes.	Almacena las traducciones de la interfaz en pares de clave-valor.	Resource Bundle (i18n)	Creare los tres archivos base: en la carpeta de recursos
Clase / Archivo	Paquete / Ruta	Qué aprender de él para tu proyecto			
login.jsp / dashboard.jsp	webapp/views/	Cómo se usan <fmt:setLocale>, <fmt:setBundle> y <fmt:message>. Idéntica técnica en tus JSP.	Son las vistas que interactúan con el usuario, usando etiquetas JSTL para mostrar datos	View (MVC)	Aplicaré las etiquetas <fmt:message> para que todo el texto sea traducido automáticamente
templates/header.jsp	webapp/views/templates/	El selector de idioma con banderas. Adaptar las 3 banderas para tu proyecto (es, en, it).	Es un fragmento de código reutilizable que contiene la navegación y el selector de idioma	Composite View / Include	Lo incluiré en todas mis páginas para no repetir código de navegación
pom.xml	raíz del proyecto	Dependencias clave: JSTL, iText, Apache POI, MySQL Driver. Agregar Jakarta Mail para OTP.	Define las librerías externas y la configuración de construcción de Maven	Dependency / Management	Aquí registraré iText JSTL y el driver de MySQL para que el proyecto funcione
vacunacion_db.sql	resources/	Estructura de tablas con FK. Guía para diseñar el esquema de saludboyaca.sql.	Contiene el script de tablas, relaciones y datos iniciales	Data Schema	Me servirá de plantilla para es base de datos

2.1 Roles en el Sistema de Referencia (Vacunación)

Rol	Módulos accesibles	Lo que debes replicar en tu proyecto
MÉDICO	Dashboard, Pacientes, Vacunas, Registros, Usuarios	Lógica de rol en AuthFilter + menú condicional según rol en header.jsp
ENFERMERO	Dashboard, Pacientes, Registros	Verificación de rol en cada Servlet antes de ejecutar acciones restringidas

Público	Solo /consulta	Ruta excluida del AuthFilter + CAPTCHA conservado
---------	----------------	---

2.2 Esquema de la Base de Datos de Referencia

Tabla	Campos — Estudia cómo se relacionan para diseñar tu propio esquema
usuarios	id, nombres, apellidos, documento, email, username, password, rol (MEDICO ENFERMERO), especialidad, institucion
pacientes	id, nombres, apellidos, documento (UNIQUE), fecha_nacimiento
vacunas	id, nombre, lote, laboratorio, fecha_vencimiento. UNIQUE KEY (lote, nombre)
registros_vacunacion	id, id_paciente (FK), id_vacuna (FK), fecha_vacunacion, id_personal_medico (FK), lugar_vacunacion, observaciones
log_cambios	id, accion (INSERT UPDATE DELETE), id_registro, id_usuario (FK), fecha

3. Internacionalización (i18n) — Está en el Código de Referencia

El sistema de vacunación ya tiene i18n completa. No hay que implementarla aquí.
El código entregado incluye 4 archivos .properties (es, en, fr, it), LocaleFilter.java,
y todos los JSP ya usan <fmt:setLocale>, <fmt:setBundle> y <fmt:message>.
Tu tarea es LEER y ENTENDER esa implementación para replicarla en SaludBoyacá
con 3 idiomas (es, en, it) y las claves propias del dominio de citas médicas.

3.1 Qué Estudiar en el Código de Referencia

Archivo a leer	Qué entender	Lo que replicarás en SaludBoyacá
LocaleFilter.java	¿Qué parámetro detecta? ¿Dónde guarda el idioma? ¿Cuándo se ejecuta?	Copia y adapta: cambia los idiomas válidos a es, en, it y quita fr
messages.properties (x4)	Estructura de claves por módulo: nav.*, login.*, dashboard.*, paciente.*	Crea 3 archivos con claves del dominio de citas: cita.*, horario.*, especialidad.*

login.jsp — encabezado	Las 2 líneas de <code>fmt:setLocale</code> + <code>fmt:setBundle</code> al inicio de cada JSP	Mismo patrón exacto en todos tus JSP de SaludBoyacá
login.jsp — selector de idioma	El bloque de banderas con <code>?lang=XX</code> en la URL	Adaptar a 3 banderas: <code>co ES · us EN · it IT</code>
dashboard.jsp	Saludo con parámetro dinámico: <code><fmt:message key='bienvenida'><fmt:param value='\${nombre}'/></fmt:message></code>	Mismo patrón para el saludo del dashboard de SaludBoyacá
templates/header.jsp	Cómo se incluye el selector de idioma en todas las páginas	Mismo mecanismo en tu header.jsp

3.2 Diferencias entre la i18n del Código de Referencia y SaludBoyacá

Aspecto	Sistema de Vacunación (referencia)	SaludBoyacá (tu proyecto)
Idiomas	es, en, fr, it (4 idiomas)	es, en, it (3 idiomas — sin francés)
Banderas en el selector	CO US FR IT	CO US IT
LocaleFilter — idiomas válidos	es en fr it	es en it (eliminar fr)
Claves de módulos	vacuna.*, registro.*, consulta.*	cita.*, horario.*, especialidad.*
Clave login.captcha	No existe (OTP reemplazó el CAPTCHA)	No existe (mismo motivo)
Aspecto	Sistema de Vacunación (referencia)	SaludBoyacá (tu proyecto)
Clave consulta.captcha	Sí existe (módulo público sin cuenta)	Sí existe (mismo caso)

Lo que Sí debes crear para SaludBoyacá (no está en el código de referencia)

✓ `messages.properties` — español con claves del dominio de citas médicas

✓ `messages_en.properties` — inglés con las mismas claves

✓ messages_it.properties — italiano con las mismas claves

✓ LocaleFilter.java — copiar del código de referencia y cambiar idiomas válidos

Las claves exactas requeridas están en la sección 9.5 de este taller.

4. Verificación OTP por Correo Electrónico

4.1 Flujo Completo del Login con OTP

El OTP (One-Time Password) agrega una segunda capa de seguridad al login. El flujo reemplaza el acceso directo al Dashboard tras validar credenciales:

Paso	Actor	Acción
1	Usuario	Accede a /login → LoginServlet.doGet() muestra login.jsp SIN CAPTCHA.
2	Usuario	Ingresa solo usuario y contraseña → POST a /login.
3	LoginServlet	Valida credenciales contra la tabla usuarios. Si falla → error en login.jsp.
4	OTPService	Genera código de 6 dígitos, lo guarda en sesión con timestamp, envía al email del usuario.
5	LoginServlet	Guarda usuario en sesión (sin el flag 'otpVerificado' aún). Redirige a /otp.
6	Usuario	Ve otp_verificacion.jsp. Revisa su correo e ingresa el código.
7	OTPServlet	Valida que el código coincida y no haya expirado (5 minutos).
8	OTPServlet	Si es válido: pone session('otpVerificado')=true → redirige a /dashboard.
9	AuthFilter	En cada petición verifica session('usuario') Y session('otpVerificado')=true.
NOTA	—	El CAPTCHA solo persiste en /consulta (consulta pública) donde no hay cuenta de usuario.

4.2 Dependencia Maven para Jakarta Mail

Agrega esta dependencia al pom.xml existente. Jakarta Mail (antigua JavaMail) permite enviar correos con Gmail SMTP sin configuraciones adicionales del servidor:

```
<!-- Agregar en la sección <dependencies> del pom.xml -->

<!-- Jakarta Mail para envío de correo OTP -->
<dependency>
  <groupId>com.sun.mail</groupId>
  <artifactId>jakarta.mail</artifactId>
  <version>2.0.1</version>
</dependency>
```

4.3 OTPService.java — Generación y Envío del Código

```
package sena.adso.captcha.util;

import jakarta.mail.*; import
jakarta.mail.internet.*; import
java.security.SecureRandom; import
java.time.Instant; import
java.util.Properties;

/**
 * Servicio para generar y enviar códigos OTP por correo electrónico.
 * Usa Gmail SMTP con autenticación de aplicación.
 */ public class
OTPService {

    // — CONFIGURAR CON TU CUENTA GMAIL —
    private static final String SMTP_HOST      = "smtp.gmail.com";    private static
    final int      SMTP_PORT      = 587;    private static final String EMAIL_REMIT    =
    "vacunacion.sena.cimm@gmail.com";    private static final String EMAIL_PASS      =
    "xxxx xxxx xxxx xxxx"; // App Password Google
    private static final int      OTP_LONGITUD = 6;    private static final
    long      OTP_EXPIRA_MS = 5 * 60 * 1000; // 5 minutos
    //

    /**
     * Genera un código OTP numérico de 6 dígitos.
     * Usa SecureRandom (criptográficamente seguro) en lugar de Random.
     */
    public static String generarOTP() {
        SecureRandom rnd = new SecureRandom();
        StringBuilder sb = new StringBuilder(OTP_LONGITUD);
        for (int i = 0; i < OTP_LONGITUD; i++) {
            sb.append(rnd.nextInt(10)); // dígito 0-9
        }
        return
        sb.toString();
    }

    /**
     * Verifica si el OTP ingresado es válido y no ha expirado.
     * @param ingresado Código que escribió el usuario
     * @param guardado Código almacenado en sesión
```

```

* @param timestamp Milisegundos Epoch en que se generó el OTP
*/
public static boolean esValido(String ingresado, String guardado, long
timestamp) {
    if (ingresado == null || guardado == null) return false;
    long ahora = Instant.now().toEpochMilli();
    boolean noExpirado = (ahora - timestamp) <= OTP_EXPIRA_MS;
    boolean coincide = ingresado.trim().equals(guardado);
    return noExpirado && coincide;
}

/**
* Envía el código OTP al correo del usuario.
* @param destinatario Email del usuario (tabla usuarios.email)
* @param codigoOTP Código generado por generarOTP()
* @param asunto Clave del asunto (vendrá del ResourceBundle en el
Servlet)
* @param cuerpo Texto del correo con el código ya interpolado
* @throws MessagingException si falla el envío
*/
public static void enviarOTP(String destinatario, String
codigoOTP, String asunto, String
cuerpo) throws MessagingException {

    Properties props = new Properties();
    props.put("mail.smtp.host", SMTP_HOST);
    props.put("mail.smtp.port", SMTP_PORT);
    props.put("mail.smtp.auth", "true");
    props.put("mail.smtp.starttls.enable", "true");

    Session mailSession = Session.getInstance(props, new Authenticator() {
        @Override protected PasswordAuthentication
getPasswordAuthentication() { return new
PasswordAuthentication(EMAIL_REMIT, EMAIL_PASS); }
    });

    Message mensaje = new MimeMessage(mailSession);
    mensaje.setFrom(new InternetAddress(EMAIL_REMIT, "Sistema
de Vacunación"));
    mensaje.setRecipient(Message.RecipientType.TO, new
InternetAddress(destinatario));
    mensaje.setSubject(asunto);
    mensaje.setText(cuerpo);

    Transport.send(mensaje);
}
}

```

4.4 Modificación de LoginServlet.java — doPost()

Solo se modifica el bloque que antes hacía sendRedirect a /dashboard. Ahora genera y envía el OTP, guarda datos en sesión y redirige a /otp:

```

// En LoginServlet.doPost() - versión NUEVA sin CAPTCHA:
// 1. Se elimina toda la lógica de CaptchaGenerator (generarTextoCaptcha,
// generarImagenCaptcha, validarCaptcha, session.setAttribute('captchaText'))

```

```
// 2. El doGet() ya NO llama a CaptchaGenerator – solo hace forward a login.jsp
```

```
@Override
```

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)  
throws ServletException, IOException {
```

```
    // Sin CAPTCHA: solo mostrar el formulario de login
```

```
    request.getRequestDispatcher("/views/login.jsp").forward(request, response);  
}
```

```
@Override
```

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)  
throws ServletException, IOException {
```

```
    String username = request.getParameter("username");
```

```
    String password = request.getParameter("password");
```

```
    // Validar credenciales directamente (sin CAPTCHA)
```

```
    UsuarioDAO usuarioDAO = new UsuarioDAO();
```

```
    Usuario usuario = usuarioDAO.validarLogin(username, password);
```

```
    if (usuario != null) {
```

```
        // ① Generar el código OTP
```

```
        String otp = OTPService.generarOTP();
```

```
        long
```

```
        timestamp = java.time.Instant.now().toEpochMilli();
```

```
        // ② Guardar en sesión: usuario y OTP (otpVerificado = false)
```

```
        HttpSession session = request.getSession();
```

```
        session.setAttribute("usuario", usuario);
```

```
        session.setAttribute("usuarioId", usuario.getId());
```

```
        session.setAttribute("usuarioNombre", usuario.getNombreCompleto());
```

```
        session.setAttribute("usuarioRol", usuario.getRol());
```

```
        session.setAttribute("otpCodigo", otp);
```

```
        session.setAttribute("otpTimestamp", timestamp);
```

```
        session.setAttribute("otpEmail", usuario.getEmail());
```

```
        session.setAttribute("otpVerificado", false);
```

```
        // ③ Obtener textos del ResourceBundle según el idioma de sesión
```

```
        String lang = (String) session.getAttribute("lang");
```

```
        if (lang ==
```

```
        null) lang = "es";
```

```
        java.util.ResourceBundle rb = java.util.ResourceBundle.getBundle(  
            "messages", new java.util.Locale(lang));
```

```
        String asunto = rb.getString("otp.email.asunto");
```

```
        String cuerpo = java.text.MessageFormat.format(  
            rb.getString("otp.email.cuerpo"), otp);
```

```
        // ④ Enviar el OTP por correo
```

```
        try {
```

```
            OTPService.enviarOTP(usuario.getEmail(), otp, asunto, cuerpo);
```

```
        } catch (Exception ex) {
```

```
            System.err.println("Error enviando OTP: " + ex.getMessage());
```

```
        }
```

```
        // ⑤ Redirigir a la página de verificación OTP
```

```
        response.sendRedirect(request.getContextPath() + "/otp");
```

```
    } else {
```

```
        // Credenciales incorrectas – sin regenerar CAPTCHA
```

```
        String lang = (String) request.getSession().getAttribute("lang");
```

```

        if (lang == null) lang = "es";
        java.util.ResourceBundle rb = java.util.ResourceBundle.getBundle(
            "messages", new java.util.Locale(lang));
        request.setAttribute("error", rb.getString("login.error.credenciales"));
        request.getRequestDispatcher("/views/login.jsp").forward(request, response);
    } }

```

4.5 OTPServlet.java — Nuevo Servlet de Verificación

```

package sena.adso.captcha.controller;

import java.io.IOException; import
java.util.ResourceBundle; import
java.util.Locale; import
javax.servlet.ServletException; import
javax.servlet.annotation.WebServlet; import
javax.servlet.http.*;
import sena.adso.captcha.util.OTPService;

@WebServlet(name = "OTPServlet", urlPatterns = {"/otp"})
public class OTPServlet extends HttpServlet {

    /** GET /otp → muestra el formulario de verificación */
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);          // Si no
        hay sesión activa, redirigir al login                      if (session == null ||
        session.getAttribute("otpCodigo") == null) {
        response.sendRedirect(request.getContextPath() + "/login");
        return;
        }
        // Pasar el email enmascarado a la vista (ej:
        adm***@gmail.com)      String email = (String)
        session.getAttribute("otpEmail");
        request.setAttribute("emailMasked", enmascararEmail(email));
        request.getRequestDispatcher("/views/otp_verificacion.jsp")
        .forward(request, response);
    }

    /** POST /otp → valida el código ingresado */
    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException,
        IOException {

        HttpSession session = request.getSession(false);
        String codigoIngres = request.getParameter("otpCodigo");
        String codigoSesion = (String) session.getAttribute("otpCodigo");
        long timestamp = (Long) session.getAttribute("otpTimestamp");

        if (OTPService.esValido(codigoIngres, codigoSesion, timestamp)) {
            // ☒ OTP correcto: marcar sesión como verificada
            session.setAttribute("otpVerificado", true);
            session.removeAttribute("otpCodigo"); // limpiar código usado

```

```

        session.removeAttribute("otpTimestamp");
response.sendRedirect(request.getContextPath() + "/dashboard");          } else {
    // ✖ OTP incorrecto o expirado
    String lang = (String) session.getAttribute("lang");
    if (lang == null) lang = "es";
    ResourceBundle rb = ResourceBundle.getBundle("messages", new
Locale(lang));
    request.setAttribute("error",
rb.getString("otp.error"));
    String email = (String)
session.getAttribute("otpEmail");
request.setAttribute("emailMasked", enmascararEmail(email));
request.getRequestDispatcher("/views/otp_verificacion.jsp")
.forward(request, response);
    }
}

/** Enmascara el email: admin@gmail.com → adm***@gmail.com */
private String enmascararEmail(String email) {
    if (email == null
|| !email.contains("@")) return "***";
    String[] partes = email.split("@");
    String local = partes[0];
    String dominio = partes[1];
    if (local.length() <= 3) return local + "***@" + dominio;
return local.substring(0, 3) + "***@" + dominio;
}
}

```

4.6 otp_verificacion.jsp — Vista de Ingreso del Código

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
<fmt:setLocale value="${sessionScope.lang != null ? sessionScope.lang : 'es'}"/>
<fmt:setBundle basename="messages"/>
<!DOCTYPE html>
<html lang="${sessionScope.lang}">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
<title><fmt:message key='otp.titulo'/></title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link
href="https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.4.0/css/all.min.css"
rel="stylesheet">
</head>
<body style="background:#f0f4f8;">
<div class="container">
    <div class="row justify-content-center mt-5">
        <div class="col-md-5">
            <div class="card shadow border-0" style="border-radius:16px;">
                <div class="card-header text-center py-4"
style="background:#0E6655;border-radius:16px 16px 0 0;">
                    <i class="fas fa-shield-alt fa-3x text-white mb-2"></i>
                    <h4 class="text-white mb-0"><fmt:message key='otp.titulo'/></h4>
                </div>
            </div>
        </div>
    </div>

```

```

<div class="card-body p-4">

    <c:if test="${not empty error}">
        <div class="alert alert-danger">${error}</div>
    </c:if>

    <p class="text-center text-muted">
        <fmt:message key='otp.instruccion'>
            <fmt:param value="${emailMasked}" />
        </fmt:message>
    </p>

    <form action="${pageContext.request.contextPath}/otp" method="post">
        <div class="mb-3">
            <label class="form-label fw-bold">
                <fmt:message key='otp.campo' />
            </label>
            <input type="text" name="otpCodigo" class="form-control
formcontrol-lg text-center" maxlength="6" pattern="[0-9]{6}"
placeholder="000000" required autofocus style="letter-
spacing:8px;font-size:1.8rem;">
        </div>
        <div class="d-grid gap-2">
            <button type="submit" class="btn btn-lg"
                style="background:#0E6655;color:white;">
                <i class="fas fa-check-circle me-2"></i>
                <fmt:message key='otp.verificar' />
            </button>
            <a href="${pageContext.request.contextPath}/login"
class="btn btn-outline-secondary">
                <fmt:message key='otp.reenviar' />
            </a>
        </div>
    </form>
</div>
</div>
</div>
</div>
</div>
</body>
</html>

```

4.7 Modificar AuthFilter.java para exigir OTP verificado

La única modificación al filtro existente es añadir la verificación del flag otpVerificado:

```

// En AuthFilter.doFilter() - reemplazar la línea de verificación:

// ANTES:
boolean isLoggedInIn = (session != null && session.getAttribute("usuario") != null);

// DESPUÉS: exige TANTO usuario como OTP verificado
boolean isLoggedInIn = (session != null
    && session.getAttribute("usuario") != null
    && Boolean.TRUE.equals(session.getAttribute("otpVerificado")));

```

```
// También agregar /otp a las URLs excluidas del filtro:
// El filtro actual protege: /dashboard /pacientes /vacunas /registros /usuarios //
/otp NO debe estar en esa lista (es accesible sin otpVerificado=true)
```

☑ Configurar Gmail para envío de OTP — App Password

1. En tu cuenta Gmail → Gestionar cuenta → Seguridad → Verificación en 2 pasos (activar).
2. En la misma sección → Contraseñas de aplicación → Crear una nueva.
3. Nombre: 'Sistema Vacunación SENA'. Google genera una clave de 16 caracteres.
4. Copiar esa clave (formato: xxxx xxxx xxxx xxxx) en OTPService.java → EMAIL_PASS.
5. NUNCA subir esa clave a GitHub. En producción usarla como variable de entorno.

En Docker: `docker run -e EMAIL_PASS='xxxx xxxx xxxx xxxx' ...` para no hardcodearla.

5. Dockerización del Sistema de Vacunación

5.1 Diferencia clave: WAR en Tomcat vs JAR Spring Boot

El aplicativo de vacunación genera un WAR (captcha.war). Para ejecutarlo en Docker necesitamos una imagen con Tomcat incluido, diferente a Spring Boot que embebe Tomcat en el JAR:

Aspecto	Spring Boot JAR	Java EE WAR (nuestro caso)
Imagen base	eclipse-temurin:17-jre	tomcat:9.0-jdk14-openjdk
Comando ejecución	java -jar app.jar	Tomcat gestiona el WAR automáticamente
Puerto interno	8080 (configurable)	8080 (Tomcat por defecto)
Copia en Dockerfile	COPY app.jar /app/app.jar	COPY captcha.war /usr/local/tomcat/webapps/
Context path	/ (raíz)	http://host:8080/captcha/ (nombre del WAR)

5.2 Dockerfile para WAR en Tomcat

```
# =====
# ETAPA 1: COMPILACIÓN - Maven + JDK 14
# =====
FROM maven:3.8.6-openjdk-14 AS build

WORKDIR /app

# Copiar pom.xml primero: si no cambia, Docker reutiliza la
# caché # de dependencias (no re-descarga en cada build) COPY
# pom.xml .
RUN mvn dependency:go-offline -B

# Copiar código fuente y compilar
COPY src ./src
RUN mvn clean package -DskipTests

# =====
# ETAPA 2: EJECUCIÓN - Tomcat 9 + JDK 14
# =====
FROM tomcat:9.0-jdk14-openjdk

# Limpiar las aplicaciones de ejemplo de Tomcat
RUN rm -rf /usr/local/tomcat/webapps/*

# Copiar el WAR compilado al directorio de Tomcat
# Tomcat lo despliega automáticamente al iniciar
COPY --from=build /app/target/captcha.war /usr/local/tomcat/webapps/ROOT.war

# Nota: usamos ROOT.war para que la app quede en la raíz
# http://host:8080/ en lugar de http://host:8080/captcha/

# Puerto de Tomcat
EXPOSE 8080

# Variables de entorno (credenciales via -e en docker run o en el hosting)
ENV
DB_URL="jdbc:mysql://TU_HOST:3306/vacunacion_db?useSSL=false&serverTimezone=UTC" ENV
DB_USER="root"
ENV DB_PASS=""
ENV EMAIL_PASS="xxxx xxxx xxxx xxxx"

# Tomcat arranca con el script estándar de la imagen base
CMD ["catalina.sh", "run"]
```

5.3 Adaptar Conexion.java para usar Variables de Entorno

Actualmente Conexion.java tiene URL, USER y PASS hardcodeados. En Docker las credenciales se pasan como variables de entorno para no exponerlas en la imagen:


```

package sena.adso.captcha.model;

import java.sql.*;

public class Conexion {

    // Leer credenciales desde variables de entorno (seguro para Docker)
    // Si la variable no existe, usar el valor local como respaldo
    private static final String URL =
        System.getenv("DB_URL") !=
        null ? System.getenv("DB_URL")
        :
        "jdbc:mysql://localhost:3306/vacunacion_db?useSSL=false&serverTimezone=UTC";

    private static final String USER =
        System.getenv("DB_USER") != null ? System.getenv("DB_USER") : "root";

    private static final String PASS =
        System.getenv("DB_PASS") != null ? System.getenv("DB_PASS") : "";

    public static Connection getConnection() throws SQLException {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection(URL, USER, PASS);
    } catch (ClassNotFoundException ex) {
        throw new
        SQLException("Error al cargar el driver de MySQL", ex);
    }
    }

    public static void closeConnection(Connection connection) {
    try {
    if (connection != null && !connection.isClosed()) connection.close();
    } catch (SQLException ex) {
        System.err.println("Error al cerrar la conexión: " + ex.getMessage());
    }
    }
    }
}

```

5.4 Archivo .dockerignore

```

target/
.git/
.gitignore
*.md
.idea/
*.iml .vscode/ nb-
configuration.xml
**/*.log

```

5.5 Secuencia Completa: Build → Test → Push

```
# 1. Compilar y construir la imagen Docker (desde la raíz del proyecto)
docker build -t tuusuario/vacunacion-sena:v1 .

# 2. Ver el tamaño de la imagen generada docker
images | grep vacunacion-sena

# 3. Ejecutar con la BD local (MySQL en la misma máquina)
# host.docker.internal apunta a localhost del sistema host en Docker Desktop
docker run -d -p 8080:8080 \
    -e DB_URL="jdbc:mysql://host.docker.internal:3306/vacunacion_db?useSSL=false&serverTimezone=UTC" \
    -e DB_USER="root" \
    -e DB_PASS="" \
    -e EMAIL_PASS="xxxx xxxx xxxx xxxx" \
    --name vacunacion-test \
    tuusuario/vacunacion-sena:v1

# 4. Verificar que Tomcat desplegó el WAR
docker logs vacunacion-test | grep -i 'deployed\|started'

# 5. Probar en: http://localhost:8080/
# Usuario: admin | Contraseña: admin123

# 6. Detener y eliminar el contenedor de prueba
docker stop vacunacion-test && docker rm vacunacion-test

# 7. Publicar en Docker Hub
docker login
docker push tuusuario/vacunacion-sena:v1
```

6. Base de Datos MySQL en la Nube — Antes de Desplegar

Los hosting gratuitos no incluyen MySQL. Necesitas una BD MySQL accesible desde internet. Estas son las opciones gratuitas más usadas para proyectos académicos:

Proveedor	Plan gratuito	Limitación	Pasos
Railway.app	500 horas/mes + 1 GB	Se suspende sin actividad	Nuevo proyecto → MySQL → copiar URL
PlanetScale	5 GB + 1 000M lecturas	No soporta FK directas	Crear DB → obtener cadena JDBC
Aiven.io	1 nodo MySQL gratis	30 días trial	Crear servicio → descargar certificado SSL
Clever Cloud	Base MySQL 256 MB	Sin acceso SSH	Dashboard → MySQL addon → credenciales
FreeSQLdb.com	Bases MySQL pequeñas	Muy limitado	Registrarse → credenciales SMTP

💡 Recomendación para este taller: Railway.app

1. Regístrate en railway.app con tu cuenta GitHub (gratis).

2. New Project → Database → MySQL.

3. En la pestaña 'Variables' copia: MYSQL_URL, MYSQL_USER, MYSQL_PASSWORD, MYSQL_HOST, MYSQL_PORT.

4. En la pestaña 'Query' ejecuta el script vacunacion_db.sql.

5. La URL JDBC quedará así:

```
jdbc:mysql://MYSQL_HOST:MYSQL_PORT/railway?useSSL=false&serverTimezone=UTC
```

6. Usar esas credenciales como variables de entorno en cada hosting.

7. Despliegue en los 5 Hosting Gratuitos

Variables de entorno comunes a todos los hosting

DB_URL =
jdbc:mysql://TU_HOST_RAILWAY:PORT/railway?useSSL=false&serverTimezone=UTC

DB_USER = TU_USUARIO_RAILWAY

DB_PASS = TU_CONTRASEÑA_RAILWAY

EMAIL_PASS= xxxx xxxx xxxx xxxx (App Password de Gmail)

7.1 Render — render.com

Característica	Detalle
Plan gratuito	750 horas/mes · 512 MB RAM · SSL · dominio .onrender.com

Limitación	Se 'duerme' tras 15 min sin peticiones — primera carga tarda ~30 s
Ideal para	Demos, sustentaciones, proyectos académicos

#	Paso
1	render.com → New → Web Service → 'Deploy an existing image from a registry'.
2	Image URL: tuusuario/vacunacion-sena:v1
3	Instance Type: Free · Region: Oregon o Frankfurt.
4	Environment: agregar DB_URL, DB_USER, DB_PASS, EMAIL_PASS.
5	Deploy Web Service. En 3-4 min: https://vacunacion-sena.onrender.com
6	Probar: login con admin/admin123 → ingresar OTP del correo → Dashboard.

7.2 Koyeb — koyeb.com

Característica	Detalle
Plan gratuito	2 servicios · 512 MB RAM · No se duerme · dominio .koyeb.app
Ventaja	Siempre activo en el plan gratuito — sin latencia de arranque

#	Paso
1	koyeb.com → Create Service → Docker.
2	Image: tuusuario/vacunacion-sena:v1 (Docker Hub público).
3	Ports: Container port = 8080, Protocol = HTTP.
#	Paso
4	Environment: DB_URL, DB_USER, DB_PASS, EMAIL_PASS.
5	Instance: nano (gratuito) → Deploy.
6	URL: https://vacunacion-sena-xxxx.koyeb.app

7.3 Google Cloud Run

Característica	Detalle
Capa gratuita	2M peticiones/mes · escala a 0 sin tráfico · SSL automático
Ventaja	Infraestructura Google, latencia mínima, sin configurar servidores

#	Paso
1	console.cloud.google.com → Cloud Run → Create Service.
2	Container image URL: docker.io/tuusuario/vacunacion-sena:v1
3	Region: us-central1 o europe-west1 · Allow unauthenticated invocations.
4	Container port: 8080 · Memory: 512 MiB.
5	Variables: DB_URL, DB_USER, DB_PASS, EMAIL_PASS.
6	Create → URL: https://vacunacion-sena-xxxx.run.app

Alternativa: desde terminal con gcloud CLI

```
gcloud auth login
gcloud config set project TU_PROJECT_ID

gcloud run deploy vacunacion-sena \
  --image docker.io/tuusuario/vacunacion-sena:v1 \
  --platform managed \
  --region us-centrall \
  --allow-unauthenticated \
  --port 8080 \
  --memory 512Mi \
  --set-env-vars DB_URL=...,DB_USER=...,DB_PASS=...,EMAIL_PASS=...
```

7.4 Oracle Cloud — Always Free

Característica	Detalle
Always Free	VM ARM Ampere A1: hasta 4 OCPU + 24 GB RAM total — el más generoso
Estrategia	VM Ubuntu + Docker instalado manualmente + ejecutar el contenedor
Característica	Detalle

Nota	Requiere tarjeta de crédito para crear la cuenta (no cobra en Always Free)
------	--

#	Paso
1	cloud.oracle.com → Compute → Instances → Create → Shape: VM.Standard.A1.Flex (1 OCPU, 6 GB).
2	OS: Ubuntu 22.04 · IP pública asignada · descargar clave SSH.
3	Networking → Security Lists → Ingress Rule: TCP puerto 8080 desde 0.0.0.0/0.
4	ssh -i clave.pem ubuntu@IP_PUBLICA
5	sudo apt update && sudo apt install docker.io -y && sudo systemctl start docker
6	sudo docker run -d -p 8080:8080 -e DB_URL=... -e DB_USER=... -e DB_PASS=... -e EMAIL_PASS=... tuusuario/vacunacion-sena:v1
7	Abrir: http://IP_PUBLICA:8080/

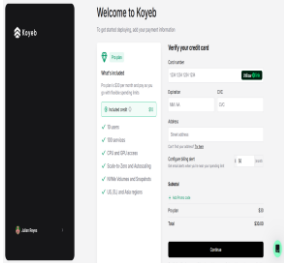
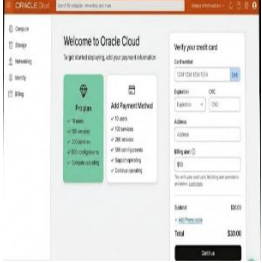
7.5 Back4App Containers — back4app.com

Característica	Detalle
Plan gratuito	1 contenedor · 256 MB RAM · 0.25 vCPU · dominio .b4a.app
Limitación clave	256 MB es justo para Tomcat — ajustar JVM: -Xms64m -Xmx200m

#	Paso
1	back4app.com → New App → Containers.
2	Docker Hub → Image: tuusuario/vacunacion-sena:v1 · Port: 8080.
3	Environment: DB_URL, DB_USER, DB_PASS, EMAIL_PASS.
4	Agregar también: JAVA_TOOL_OPTIONS=-Xms64m -Xmx200m (para no superar 256 MB RAM).
5	Plan: Sandbox → Create App → URL: https://vacunacion-sena.b4a.app

8. Tabla Comparativa de los 5 Hosting

Completa esta tabla durante la sesión de despliegue. Registra los datos reales que obtienes.

Criterio	Render	Koyeb	Cloud Run	Oracle	Back4App
RAM disponible	512 MB	512 MB	512 MB	Hasta 6 GB	256 MB
Se duerme	Sí (15 min)	No	Sí (escala 0)	No (VM activa)	No
Tarjeta requerida	No	No	No	Sí (sin cobro)	No
Dificultad	Baja	Baja	Media	Alta	Baja
Dominio gratuito	.onrender.com	.koyeb.app	.run.app	IP pública	.b4a.app
SSL automático	Sí	Sí	Sí	No (manual)	Sí
Tiempo de deploy	3-4 min	1-2 min	2-3 min	10-15 min	4-6 min
Tu URL obtenida	https://saludboyaca-app-latest-1.onrender.com				https://saludboyaca-app-latest-1.b4a.app
Tiempo real	9 segundos	Pide pagar para poder acceder	Este también pide pagar para poder acceder y da errores	Este igual me pide pagar para poder acceder	11 segundos

9. Proyecto Final — Nueva Aplicación Java Web

¿Por qué una aplicación NUEVA y no modificar la de vacunación?

El taller de i18n, OTP y Docker se practicó sobre el sistema de vacunación que ya conoces.

El proyecto final demuestra que comprendes los conceptos, no que seguiste instrucciones.

Construir una aplicación diferente desde cero —con i18n, OTP y Docker integrados desde

el diseño— es la evidencia real de aprendizaje autónomo que exige la formación ADSO.

9.1 Descripción del Sistema — SaludBoyacá: Citas Médicas

El Centro de Salud Municipal de Paipa, Boyacá, necesita un sistema web para gestionar citas médicas con acceso seguro para el personal clínico (MÉDICO, RECEPCIONISTA, ENFERMERO) y una ventanilla pública donde los pacientes consulten y descarguen su comprobante de cita sin necesidad de cuenta. El sistema debe estar en español, inglés e italiano, con verificación OTP por correo al iniciar sesión y publicado en al menos dos hostings gratuitos.

Ficha del Proyecto Final — SaludBoyacá	
Nombre del sistema:	SaludBoyacá — Gestión de Citas Médicas
Entidad:	Centro de Salud Municipal de Paipa, Boyacá
Dominio:	Salud pública / atención primaria
Idiomas requeridos:	Español (por defecto), English, Italiano
Autenticación:	Solo usuario + contraseña → OTP por correo (sin CAPTCHA en login)
Módulo público:	Consulta de cita por número de documento · CAPTCHA visual (único mecanismo posible sin cuenta de usuario)
Tecnología:	Java EE · Servlets · JSP · JSTL · MySQL · Jakarta Mail · Docker
Despliegue:	Imagen en Docker Hub + mínimo 2 hosting de los 5 estudiados
Modalidad:	Individual o parejas (máximo 2 aprendices)
Duración:	3 semanas de trabajo autónomo fuera del laboratorio

9.2 Modelo de Datos — Esquema Completo

El sistema debe tener exactamente estas 7 tablas. El script SQL completo (CREATE TABLE + datos de prueba reales de Paipa) es el primer entregable.


```

-- ① Tabla de usuarios del sistema (personal clínico)
CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY, nombres
    VARCHAR(80) NOT NULL, apellidos VARCHAR(80) NOT NULL,
    documento VARCHAR(20) NOT NULL UNIQUE, email VARCHAR(100)
    NOT NULL UNIQUE, username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL, rol
    ENUM('MEDICO','RECEPCIONISTA','ENFERMERO') NOT NULL, especialidad
    VARCHAR(80), lang_preferido VARCHAR(5) DEFAULT 'es', activo
    TINYINT(1) DEFAULT 1);

-- ② Pacientes (personas que solicitan citas)
CREATE TABLE pacientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombres VARCHAR(80) NOT NULL, apellidos
    VARCHAR(80) NOT NULL, documento VARCHAR(20)
    NOT NULL UNIQUE, fecha_nacimiento DATE NOT
    NULL, telefono VARCHAR(20), email
    VARCHAR(100), eps VARCHAR(80) NOT
    NULL, vereda_barrio VARCHAR(80));

-- ③ Especialidades médicas disponibles
CREATE TABLE especialidades (
    id INT AUTO_INCREMENT PRIMARY KEY,

```

```

    nombre          VARCHAR(80)  NOT NULL,
descripcion        VARCHAR(200) );

```

```
-- ④ Horarios disponibles por médico CREATE
```

```

TABLE horarios (
    id              INT AUTO_INCREMENT PRIMARY KEY,
id_medico          INT NOT NULL,
    dia_semana      TINYINT NOT NULL COMMENT '1=Lun 2=Mar 3=Mié 4=Jue
5=Vie',    hora_inicio  TIME NOT NULL,    hora_fin      TIME NOT NULL,
max_citas          INT DEFAULT 10,
    FOREIGN KEY (id_medico) REFERENCES usuarios(id)
);

```

```
-- ⑤ Citas médicas (entidad central del sistema)
```

```

CREATE TABLE citas (
    id              INT AUTO_INCREMENT PRIMARY
KEY,    id_paciente    INT NOT NULL,
id_medico          INT NOT NULL,    id_especialidad
INT NOT NULL,    fecha_cita    DATE    NOT
NULL,    hora_cita      TIME    NOT NULL,
motivo             VARCHAR(300),
    estado          ENUM('PROGRAMADA','CONFIRMADA','ATENDIDA','CANCELADA') DEFAULT
'PROGRAMADA',
    observaciones   VARCHAR(500),
    fecha_registro  DATETIME    DEFAULT    CURRENT_TIMESTAMP,
id_registrado_por INT,
    FOREIGN KEY (id_paciente)    REFERENCES pacientes(id),
    FOREIGN KEY (id_medico)      REFERENCES usuarios(id),
    FOREIGN KEY (id_especialidad) REFERENCES especialidades(id)
);

```

```
-- ⑥ Tokens OTP (seguridad de autenticación)
```

```

CREATE TABLE otp_tokens (
    id              INT AUTO_INCREMENT PRIMARY KEY,
id_usuario          INT NOT NULL,    codigo
VARCHAR(6)    NOT NULL,    fecha_gen    DATETIME
DEFAULT CURRENT_TIMESTAMP,    expira_en    DATETIME
NOT NULL,    usado      TINYINT(1)    DEFAULT 0,
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id)
);

```

```
-- ⑦ Log de accesos (auditoría) CREATE
```

```

TABLE log_accesos (
    id              INT AUTO_INCREMENT PRIMARY KEY,
id_usuario          INT,    username    VARCHAR(50),
accion             VARCHAR(50) NOT NULL,    ip
VARCHAR(45),    resultado    ENUM('EXITO','FALLO') NOT
NULL,    fecha            DATETIME    DEFAULT
CURRENT_TIMESTAMP );

```

```
-- === DATOS DE PRUEBA - contexto real de Paipa, Boyacá ===
```

```

INSERT INTO especialidades (nombre) VALUES
('Medicina General'), ('Odontología'), ('Pediatría'),
('Ginecología'), ('Optometría');

```

```
INSERT INTO usuarios (nombres, apellidos, documento, email, username, password,
rol, especialidad) VALUES
('Carlos Ernesto', 'Pedraza Rondón', '1052345678',
'cpedraza@saludboyaca.gov.co', 'cpedraza', 'admin123', 'MEDICO',
'Medicina General'),
('María Eugenia', 'Suárez Cely', '1052345679',
'msuarez@saludboyaca.gov.co', 'msuarez', 'enfermero1', 'ENFERMERO', NULL),
('Jorge Hernando', 'Báez Morales', '1052345680',
'jbaez@saludboyaca.gov.co', 'jbaez', 'recepl23', 'RECEPCIONISTA', NULL);
```

9.3 Roles y Permisos — Especificación Detallada

Rol	Acceso	Permisos detallados
MÉDICO	Alto	Dashboard con sus propias estadísticas · Ver y gestionar sus citas (confirmar, marcar atendida, cancelar) · Ver pacientes · Ver su horario · Exportar lista de citas del día en PDF.
RECEPCIONISTA	Medio	Dashboard general · Pacientes (CRUD completo) · Citas (CRUD completo: programar, editar, cancelar) · Horarios (solo lectura) · Generar comprobante de cita en PDF.
ENFERMERO	Básico	Dashboard · Citas (solo lectura: ver listado y detalle) · Pacientes (solo lectura). NO puede crear ni modificar citas ni pacientes.
PÚBLICO (sin sesión)	Consulta	Solo accede a /consulta-cita: busca por número de documento. No tiene cuenta ni correo → usa CAPTCHA visual (el único mecanismo anti-bot disponible aquí). Ve sus próximas citas y descarga comprobante PDF.

Regla de AuthFilter para SaludBoyacá

El AuthFilter protege: /dashboard /pacientes /citas /horarios /usuarios

Condición de acceso: session('usuario') != null AND session('otpVerificado') = true

La ruta /otp y /login están excluidas del filtro.

La ruta /consulta-cita es pública (sin sesión).

La ruta /api/horarios-disponibles es pública (para el selector de citas).

9.4 Arquitectura Java — Paquetes y Clases Requeridas

```
src/main/java/co/sena/cimm/adso/saludboyaca/
| └─
model/
|   └─ Conexion.java           ← lee DB_URL, DB_USER, DB_PASS de env
vars | └─ dto/ | └─ Usuario.java   ← id, nombres,
apellidos, email, rol, langPref
|   └─ Paciente.java           ← id, nombres, apellidos, documento, eps,
etc.
|   └─ Especialidad.java       ← id, nombre, descripcion
|   └─ Horario.java           ← id, idMedico, diaSemana, horaInicio,
horaFin
|   └─ Cita.java               ← id, paciente, medico, especialidad,
fecha, estado | └─ dao/
|   └─ UsuarioDAO.java         ← validarLogin(), buscarPorId(),
listarTodos()
|   └─ PacienteDAO.java        ← insertar(), listar(),
buscarPorDocumento()
|   └─ EspecialidadDAO.java    ← listarTodas() |
└─ HorarioDAO.java             ← listarPorMedico(),
horasDisponibles(fecha)
|   └─ CitaDAO.java           ← insertar(), listar(), cambiarEstado(),
eliminar()
|   └─ OTPTokenDAO.java        ← insertar(), validar(), marcarUsado()
|   └─ servlet/ | └─ LoginServlet.java   ← GET: muestra login
POST: valida → genera OTP
|   └─ OTPServlet.java         ← GET: muestra formulario · POST: valida
código
|   └─ DashboardServlet.java   ← estadísticas según el rol
|   └─ PacienteServlet.java    ← CRUD pacientes
|   └─ CitaServlet.java        ← CRUD citas (con validación de
disponibilidad)
|   └─ HorarioServlet.java     ← solo lectura para RECEPCIONISTA/MÉDICO
|   └─ ConsultaCitaServlet.java ← módulo público (CAPTCHA + búsqueda)
|   └─ LogoutServlet.java      ← invalida sesión, redirige a /login
|   └─
util/
|   └─ LocaleFilter.java       ← @WebFilter('//*'), detecta ?lang=XX
|   └─ AuthFilter.java         ← @WebFilter('/dashboard/*',
'/pacientes/*', ...)
|   └─ OTPService.java         ← generarOTP(), esValido(), enviarOTP()
└─ CaptchaGenerator.java      ← SOLO en ConsultaCitaServlet (NO en login, NO
en dashboard)
   └─ PDFGenerator.java       ← comprobante de cita en PDF

src/main/resources/
└─ messages.properties        ← español (por defecto)
└─ messages_en.properties    ← inglés
└─ messages_it.properties    ← italiano

src/main/webapp/
└─ WEB-INF/web.xml └─
views/
└─ login.jsp
```

```

├─ otp_verificacion.jsp
├─ dashboard.jsp
├─ pacientes/           ← lista.jsp   formulario.jsp
├─ citas/               ← lista.jsp   formulario.jsp   detalle.jsp
├─ horarios/           ← lista.jsp
├─ consulta_cita.jsp
├─ templates/header.jsp ← nav con selector de idioma
└─ error.jsp

```

9.5 Internacionalización — Claves Requeridas en los .properties

El sistema debe tener estas claves mínimas en los 3 idiomas. Claves faltantes o en un solo idioma se penalizan en la rúbrica.

```

# --- GENERAL ---
app.nombre=SaludBoyacá - Gestión de Citas
app.institucion=Centro de Salud Municipal de Paipa,
Boyacá app.footer=SENA · CIMM · Regional Boyacá · 2026
app.lang.seleccionar=Idioma app.lang.es=Español
app.lang.en=English app.lang.it=Italiano

# --- LOGIN Y OTP ---
login.titulo=Iniciar Sesión login.usuario=Usuario
login.contrasena=Contraseña login.ingresar=Ingresar
login.error.credenciales=Usuario o contraseña incorrectos
otp.titulo=Verificación en Dos Pasos
otp.instruccion=Ingresa el código de 6 dígitos enviado a: {0}
otp.campo=Código OTP otp.verificar=Verificar otp.reenviar=Reenviar
código otp.error=Código incorrecto o expirado. Intente de nuevo.
otp.email.asunto=Código de verificación - SaludBoyacá
otp.email.cuerpo=Su código de verificación es: {0} (válido 5
minutos)

# --- NAVEGACIÓN ---
nav.dashboard=Panel de Control nav.pacientes=Pacientes nav.citas=Citas
Médicas nav.horarios=Horarios nav.salir=Cerrar Sesión
nav.consulta=Consulta de Cita

# --- DASHBOARD ---
dashboard.bienvenida=Bienvenido, {0} dashboard.citas.hoy=Citas para hoy
dashboard.citas.pendientes=Citas pendientes dashboard.citas.mes=Citas
este mes
dashboard.pacientes.total=Total de pacientes

# --- PACIENTES ---
paciente.titulo=Gestión de Pacientes paciente.nuevo=Nuevo Paciente
paciente.nombres=Nombres paciente.apellidos=Apellidos

```

```

paciente.documento=Número de Documento
paciente.nacimiento=Fecha de Nacimiento paciente.eps=EPS /
Aseguradora paciente.telefono=Teléfono
paciente.guardar=Guardar paciente.cancelar=Cancelar
paciente.editar=Editar paciente.eliminar=Eliminar
paciente.confirmar=¿Está seguro de eliminar este
paciente?

# — CITAS

cita.titulo=Gestión de Citas cita.nueva=Nueva Cita
cita.paciente=Paciente cita.medico=Médico
cita.especialidad=Especialidad cita.fecha=Fecha de la Cita
cita.hora=Hora
cita.motivo=Motivo de consulta cita.estado=Estado
cita.estado.programada=Programada
cita.estado.confirmada=Confirmada
cita.estado.atendida=Atendida
cita.estado.cancelada=Cancelada
cita.descargar=Descargar Comprobante PDF

# — CONSULTA PÚBLICA
consulta.titulo=Consulta de Cita Médica
consulta.instruccion=Ingresa su número de documento para ver sus citas
programadas consulta.documento=Número de documento consulta.captcha=Código de
verificación
consulta.captcha.error=El código CAPTCHA ingresado es incorrecto
consulta.buscar=Consultar
consulta.descargar=Descargar Comprobante
consulta.no.encontrado=No se encontraron citas con ese número de documento

# — ERRORES
error.requerido=Este campo es requerido
error.acceso=No tiene permisos para acceder a esta sección
error.cita.no.disponible=El médico no tiene disponibilidad en esa fecha y hora
error.servidor=Error interno. Contacte al administrador.

```

9.6 Paleta de Colores y Guía de Diseño Visual

El sistema SaludBoyacá tiene su propia identidad visual, diferente al sistema de vacunación. Se basa en azul institucional salud + verde SENA como acento. El aprendiz NO puede reutilizar los estilos del sistema de vacunación — debe crear un styles.css propio.

Elemento de UI	Nombre del color	Código HEX	Dónde se usa
Color primario	Azul salud	#1A5276	Navbar, botones primarios, encabezados de módulo

Color secundario	Verde SENA	#39A900	Botones de éxito, badges de estado 'Confirmada', ícono dashboard
Elemento de UI	Nombre del color	Código HEX	Dónde se usa
Acento institucional	Celeste suave	#2E86C1	Hover de menú, bordes de tarjetas, íconos informativos
Fondo body	Gris hielo	#EAF0F7	Background de todas las páginas internas
Tarjetas / Forms	Blanco puro	#FFFFFF	Cards, modales, formularios, tablas
Navbar texto	Blanco	#FFFFFF	Todos los textos dentro de la barra de navegación
Texto títulos	Azul oscuro	#154360	Títulos h1, h2 de módulos
Texto normal	Gris oscuro	#2C3E50	Párrafos, etiquetas de formulario
Texto suave	Gris medio	#7F8C8D	Placeholders, pie de página, ayudas de campo
Estado: Programada	Amarillo ámbar	#F39C12	Badge en tabla de citas
Estado: Confirmada	Verde salud	#27AE60	Badge en tabla de citas
Estado: Atendida	Azul info	#2980B9	Badge en tabla de citas
Estado: Cancelada	Rojo suave	#E74C3C	Badge en tabla de citas
Alerta éxito	Verde claro	#D5F5E3	Fondo de alertas de operación exitosa
Alerta error	Rojo claro	#FADBD8	Fondo de alertas de error/validación
OTP / Seguridad	Morado institucional	#6C3483	Pantalla OTP: fondo del card-header, ícono escudo
Login card	Azul muy claro	#D6EAF8	Fondo del card de login

CSS personalizado requerido (no solo Bootstrap)

El aprendiz debe crear src/main/webapp/resources/css/saludboyaca.css con al menos:

- Variables CSS (--color-primario: #1A5276; --color-sena: #39A900; etc.)

- Clase .navbar-saludboyaca con fondo #1A5276 y texto blanco

- Clase .card-stat con sombra suave, borde izquierdo de 4px en color por módulo

- Clase .badge-estado-* para los 4 estados de cita con colores definidos arriba

- Clase .otp-card con borde superior morado de 4px y fondo blanco

- Clase .btn-saludboyaca con fondo #1A5276, texto blanco, hover #154360

- Clase .login-wrapper con fondo gradiente suave azul → celeste

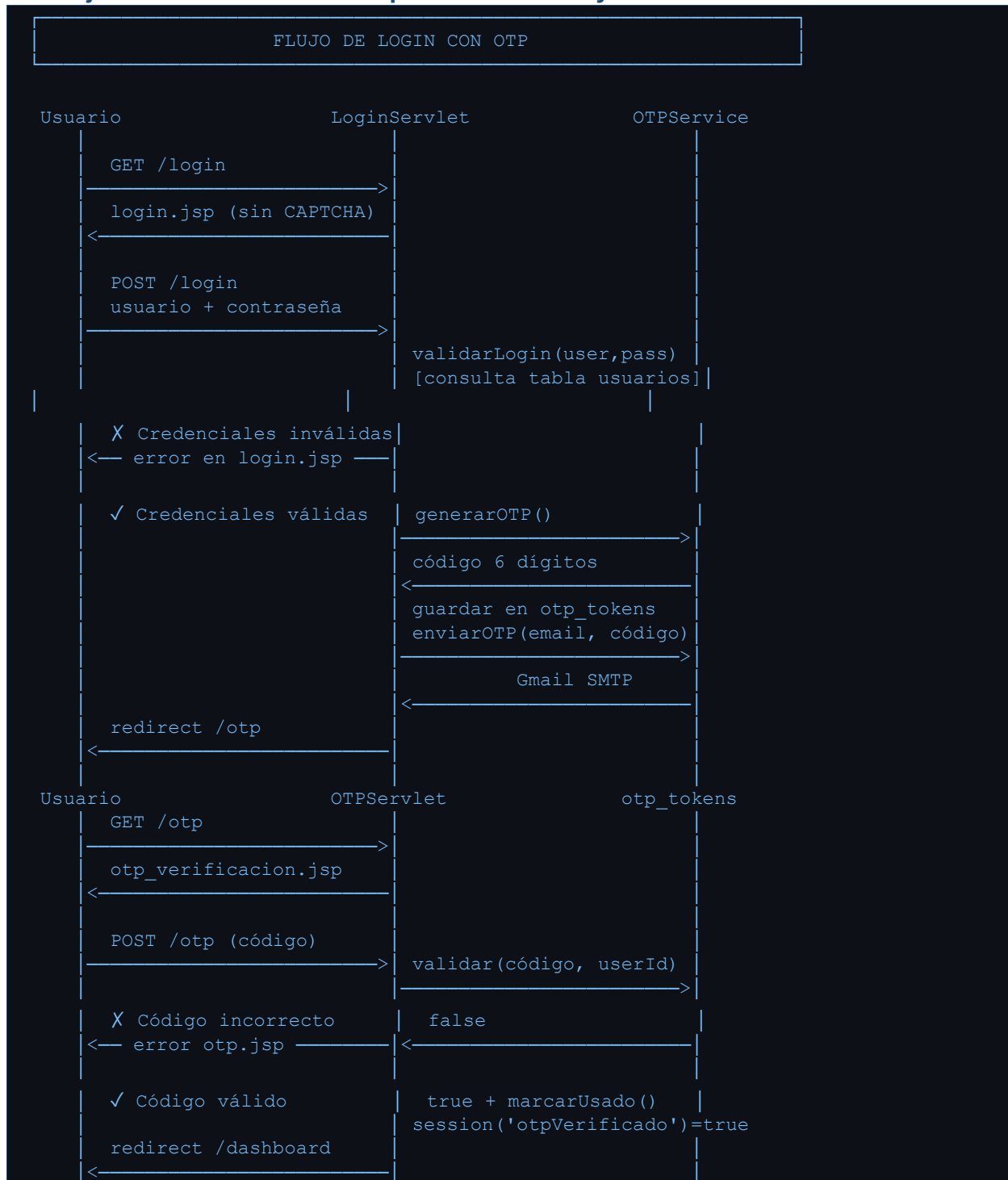
NO se puede copiar styles.css del proyecto de vacunación.

9.7 Diseño de Pantallas — Especificación Visual por Módulo

Pantalla	Especificación de elementos de UI
Login	• Fondo: gradiente diagonal de #1A5276 a #2E86C1 • Card centrado (maxwidth 420px) con fondo #D6EAF8 y border-radius 16px • Logo del centro de salud arriba (imagen o texto estilizado) • Campos usuario y contraseña con ícono FontAwesome • Botón 'Ingresar' full-width en #1A5276 • Selector de 3 idiomas con banderas (co ES · us EN · it IT) • Link '¿Olvidó su contraseña?' y link 'Consulta de cita' al pie • SIN CAPTCHA
OTP Verificación	• Fondo mismo gradiente del login • Card centrado (max-width 420px) • Cardheader con fondo #6C3483 y ícono fa-shield-alt en blanco • Email del usuario enmascarado (adm***@saludboyaca.gov.co) • Campo OTP: font-size 2rem, letter-spacing 12px, text-align center, max-length 6 • Contador regresivo 5:00 en rojo cuando quedan < 60 segundos • Botón 'Verificar' en #6C3483 full-width • Link 'Reenviar código' con contador de espera
Dashboard	• Navbar #1A5276: logo + menú según rol + selector idioma + nombre usuario + logout • 4 tarjetas métricas en fila: Citas Hoy (azul), Pendientes (ámbar), Mes (verde), Pacientes (celeste) • Cada tarjeta tiene ícono FontAwesome grande, número prominente y etiqueta traducida • Tabla 'Próximas citas' con columnas: Hora, Paciente, Especialidad, Estado (badge) • Accesos rápidos según rol: botones de Nueva Cita, Nuevo Paciente

Gestión de Citas	• Barra superior con título traducido + botón 'Nueva Cita' • DataTable con columnas: Fecha, Hora, Paciente, Médico, Especialidad, Estado, Acciones • Columna Estado: badge coloreado según el ENUM (Programada=ámbar, Confirmada=verde, etc.) • Botones de acción: Detalle (azul), Cambiar estado (verde), Cancelar (rojo) • Filtro por fecha y por estado en la parte superior • Exportar a PDF solo para MÉDICO y RECEPCIONISTA
Formulario Nueva Cita	• Layout en 2 columnas (Bootstrap grid col-md-6) • Select de paciente con búsqueda (filtrado por nombre/documento en tiempo real opcional) • Select de especialidad → al cambiar carga los médicos de esa especialidad (JS + AJAX o recarga) • Date picker para la fecha • Select de hora: carga solo las horas disponibles según médico + fecha • Campo de motivo (textarea) • Botones: Programar Cita (azul) y Cancelar (gris)
Consulta Pública	• Página sin navbar ni autenticación • Header simple con logo y nombre del centro de salud • Card centrado con campo de documento + CAPTCHA visual (el ciudadano no tiene cuenta ni correo → no es posible enviar OTP aquí) • Resultado: tabla con citas del paciente (fecha, hora, médico, especialidad, estado) • Botón 'Descargar Comprobante PDF' para cada cita Programada o Confirmada • Mensaje si no hay citas: texto con clave i18n consulta.no.encontrado • Selector de idioma visible (3 opciones)

9.8 Flujo de Autenticación Completo — SaludBoyacá



9.9 Dockerización y Despliegue

El proyecto debe empaquetarse como WAR y dockerizarse exactamente con la misma estrategia del taller (Multi-Stage Build con Tomcat 9). Las variables de entorno son las mismas, solo cambia el nombre del WAR:

```
# Dockerfile para SaludBoyacá (idéntica estructura al taller)
FROM maven:3.8.6-openjdk-14 AS build
WORKDIR /app COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

FROM tomcat:9.0-jdk14-openjdk
RUN rm -rf /usr/local/tomcat/webapps/*
COPY --from=build /app/target/saludboyaca-1.0.war
/usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080

ENV DB_URL="jdbc:mysql://HOST:3306/saludboyaca?useSSL=false&serverTimezone=UTC" ENV
DB_USER="root"
ENV DB_PASS=""
ENV EMAIL_PASS="xxxx xxxx xxxx xxxx"

CMD ["catalina.sh", "run"]
```

Entregable de despliegue	Criterio de aceptación
Imagen en Docker Hub	URL pública: hub.docker.com/r/tuusuario/saludboyaca visible sin login
Hosting 1 (obligatorio)	Render o Koyeb. URL funcional. Flujo completo: login → OTP → dashboard → cita
Hosting 2 (obligatorio)	Diferente al hosting 1. Misma verificación de flujo completo
Hosting 3 (opcional +0.3)	Cualquiera de los otros 3 estudiados. URL funcional

9.10 Entregables y Rúbrica de Evaluación

Entregables obligatorios

- Repositorio GitHub público (o archivo ZIP) con el código fuente completo.
- Script SQL en src/main/resources/saludboyaca.sql (CREATE + datos de prueba reales de Paipa).
- Dockerfile en la raíz del proyecto.
- README.md con: URLs de Docker Hub y hosting desplegados, instrucciones de ejecución local, credenciales de prueba, capturas de pantalla de las 6 pantallas principales.
- Video demostración (3-5 minutos): login → OTP por correo → cambio de idioma → crear cita → consulta pública → descargar PDF. Subido a YouTube (no listado) o Google Drive.

Rúbrica de Evaluación — 100 puntos

#	Criterio de Desempeño	Puntos	Instrumento
---	-----------------------	--------	-------------

1	Script SQL con las 7 tablas correctas (FK, ENUM, datos reales de Paipa)	8	Revisión SQL
2	Arquitectura de paquetes correcta (dto, dao, servlet, util) — cero SQL en Servlets	10	Revisión código
#	Criterio de Desempeño	Puntos	Instrumento
3	i18n funcional en 3 idiomas (es, en, it) — cero textos hardcodeados en JSP	12	Demostración
4	LocaleFilter detecta ?lang=XX y persiste el idioma en sesión durante toda la navegación	8	Demostración
5	OTPService genera código seguro + lo envía al correo + lo guarda en otp_tokens	12	Demostración
6	Flujo completo: Login (sin CAPTCHA) → OTP → Dashboard. AuthFilter exige otpVerificado	10	Demostración
7	CRUD de citas con validación de disponibilidad + cambio de estado + exportar PDF	12	Demostración
8	Consulta pública: CAPTCHA visual (no hay OTP posible sin cuenta) + resultado en idioma activo + descargar comprobante	8	Demostración
9	CSS propio (saludboyaca.css) con variables, paleta definida, sin copiar styles.css	8	Revisión CSS
10	Dockerfile funcional + imagen en Docker Hub + desplegado en 2 hosting con URL activa	12	Revisión URLs
BONUS A	Tercer hosting desplegado con URL funcional	+0,3	Revisión URL
BONUS B	Contador regresivo JS en OTP + bloqueo tras 3 intentos fallidos	+0,3	Demostración
BONUS C	Select de hora disponible carga dinámicamente con JavaScript/AJAX	+0,4	Demostración

 Condiciones de NO aprobación automática

- SQL hardcodeado dentro de Servlets (lógica de BD mezclada con control HTTP).
- PreparedStatement sustituido por concatenación de Strings (riesgo SQL Injection).
- Aplicación que no compila o no despliega en Tomcat local.
- i18n implementado con textos hardcodeados en JSP (no usa <fmt:message>).
- OTP en el login reemplazado por CAPTCHA (confunde los dos mecanismos).
- CSS copiado directamente de styles.css del sistema de vacunación.
- Video demostración que no muestra el flujo OTP completo.

10. Ejercicios Prácticos por Sesión

Distribución del taller — 4 sesiones de laboratorio + trabajo autónomo

Sesión 1 (4-5 h): Análisis del código de vacunación — estudiar i18n, LocaleFilter,

estructura Servlet+DAO, roles y JSP del sistema de referencia.

Sesión 2 (4-5 h): OTP + Docker — estudiar OTPService en el código de referencia,

dockerizar el sistema de vacunación y desplegarlo en 2 hosting.

Sesión 3 (4-5 h): Inicio SaludBoyacá — crear el proyecto Maven, la BD, los DTOs,

los DAOs, el LoginServlet con OTP y la vista de verificación.

Trabajo autónomo (3 semanas): completar SaludBoyacá — módulos CRUD, i18n,

consulta pública, CSS propio, Dockerfile y despliegue en 2 hosting.

10.1 Sesión 1 — Análisis del Código de Referencia (Vacunación)

Objetivo: entender cómo funciona cada componente del sistema de vacunación ANTES de construir el propio. Todo lo que aprendas aquí se replica en SaludBoyacá.

#	Actividad
1	Importa el proyecto captcha en tu IDE. Compila con mvn clean package y despliega en Tomcat. Accede a http://localhost:8080/captcha/ , inicia sesión con admin/admin123 y navega todos los módulos.
2	Abre LocaleFilter.java. Lee el código línea por línea y responde en tu bitácora: ¿qué parámetro detecta? ¿dónde guarda el idioma? ¿qué pasa si no hay idioma en sesión? ¿cuándo se ejecuta este filtro?
3	Abre messages.properties, messages_en.properties, messages_fr.properties y messages_it.properties. Compara las claves. Cambia el idioma en el navegador con ?lang=en y verifica que los textos cambian. Luego prueba ?lang=it.
4	Abre login.jsp. Localiza las etiquetas <fmt:setLocale>, <fmt:setBundle> y todos los <fmt:message key='...'/>. Dibuja en tu bitácora el mapa completo: qué clave va en qué elemento HTML.
5	Abre LoginServlet.java. Traza el flujo doGet() → doPost() en tu bitácora. Identifica: ¿dónde valida credenciales? ¿qué guarda en sesión? ¿a qué URL redirige? Localiza las líneas exactas del bloque CAPTCHA que vas a ELIMINAR en la sesión 2.
6	Abre AuthFilter.java. ¿Cuáles rutas protege? ¿Qué condición verifica? ¿A dónde redirige si no hay sesión? Anota la línea exacta que debes modificar para agregar la verificación OTP.
7	Abre PacienteServlet.java. Identifica el patrón switch/acción (listar, nuevo, editar, eliminar). Dibuja el diagrama de flujo completo en tu bitácora. Este mismo patrón lo usarás para CitaServlet en SaludBoyacá.
8	Abre PDFGenerator.java. ¿Qué librería usa? ¿Qué datos incluye en el certificado? Anota qué necesitas adaptar para generar el comprobante de cita de SaludBoyacá.

10.2 Sesión 2 — OTP + Docker sobre el Sistema de Vacunación

Objetivo: practicar la implementación de OTP y Docker sobre el código conocido, antes de aplicarlos en SaludBoyacá. Al final de esta sesión tendrás el sistema de vacunación dockerizado y desplegado.

#	Actividad
1	Agrega la dependencia de Jakarta Mail al pom.xml del sistema de vacunación (sección 4.2). Ejecuta mvn clean install y verifica que compila sin errores.
2	Crea OTPService.java en util/ (sección 4.3). Configura tu cuenta Gmail con App Password (sección 4.7). Prueba enviar un correo con un método main() temporal antes de integrarlo al sistema.

3	Modifica LoginServlet.java: elimina el bloque completo del CAPTCHA en doGet() y doPost(). Agrega la generación y envío de OTP al final del bloque de credenciales válidas (sección 4.4). Asegúrate de actualizar el email en la tabla usuarios del usuario admin.
4	Elimina el bloque CAPTCHA de login.jsp (div.captcha-container + input captcha). Crea otp_verificacion.jsp (sección 4.6). Crea OTPServlet.java (sección 4.5).
5	Modifica AuthFilter.java para exigir otpVerificado=true (sección 4.7). Prueba el flujo completo: login (solo usuario + contraseña) → correo con código → ingresar OTP → Dashboard.
6	Adapta Conexion.java para leer credenciales desde variables de entorno (sección 5.3). Crea el Dockerfile con Multi-Stage Build (sección 5.2). Construye la imagen: docker build -t tuusuario/vacunacion-sena:v1 .
7	Ejecuta el contenedor localmente con las variables de Railway y prueba el flujo OTP completo dentro del Docker. Sube la imagen a Docker Hub: docker push tuusuario/vacunacion-sena:v1
8	Despliega en Render y en Koyeb (secciones 7.1 y 7.2). Verifica que el flujo OTP funciona en producción. Completa la tabla comparativa (sección 8).

10.3 Sesión 3 — Inicio del Proyecto SaludBoyacá

Objetivo: crear el esqueleto del proyecto final con los componentes críticos funcionando: BD, Conexion.java, LoginServlet con OTP y otp_verificacion.jsp. Al salir de laboratorio el login debe funcionar.

#	Actividad
1	Crea el proyecto Maven en NetBeans/IntelliJ con groupId co.sena.cimm.adso.saludboyaca y packaging WAR. Copia el pom.xml del sistema de vacunación y ajusta las dependencias (agrega Jakarta Mail, conserva JSTL, iText, MySQL Driver).
2	Ejecuta el script saludboyaca.sql (sección 9.2) en MySQL local. Verifica que las 7 tablas se crean correctamente y que los datos de prueba de Paipa aparecen.
3	Crea Conexion.java adaptado de la referencia pero con lectura de variables de entorno. Crea los 5 DTOs: Usuario.java, Paciente.java, Especialidad.java, Horario.java, Cita.java.
4	Crea UsuarioDAO.java con los métodos validarLogin() y buscarPorId(). Crea OTPTokenDAO.java con insertar(), validar() y marcarUsado(). Prueba validarLogin() con un método main() temporal.
#	Actividad
5	Crea LocaleFilter.java (copia y adapta el del sistema de vacunación — es reutilizable casi sin cambios). Crea los 3 archivos .properties con las claves de la sección 9.5.

6	Crea LoginServlet.java con el flujo sin CAPTCHA: validar credenciales → generar OTP → enviar por correo → redirigir a /otp. Usa como referencia el LoginServlet modificado del sistema de vacunación.
7	Crea login.jsp con i18n (fmt:message), gradiente azul #1A5276, selector de 3 idiomas (co us π) y SIN CAPTCHA. Crea otp_verificacion.jsp con diseño morado y campo OTP grande.
8	Crea OTPServlet.java y AuthFilter.java. Prueba el flujo completo: login → OTP → (si funciona) muestra una página temporal /dashboard que diga 'Dashboard en construcción'. Presenta al instructor antes de salir.

☒ Punto de control al final de la sesión 3

El instructor verifica que el aprendiz puede:

1. Acceder a <http://localhost:8080/saludboyaca/> y ver el login.jsp en 3 idiomas.
2. Ingresar usuario y contraseña → recibir el código OTP en el correo.
3. Ingresar el código → acceder a la página temporal del dashboard.
4. Intentar acceder a /dashboard sin OTP → ser redirigido al login.

Si no se supera este punto de control, el trabajo autónomo no puede comenzar.

10.4 Trabajo Autónomo — 3 Semanas para Completar SaludBoyacá

Con el esqueleto funcionando, completa el proyecto en 3 semanas siguiendo este plan de trabajo. El instructor revisará el avance al inicio de cada semana.

Semana	Módulos a completar	Referencia en el código de vacunación	Entregable de control
Semana 1	PacienteServlet + CRUD pacientes + pacientes/lista.jsp + pacientes/formulario.jsp	PacienteServlet.java + pacientes.jsp + paciente_form.jsp	Demo CRUD pacientes funcional con i18n

Semana 2	CitaServlet + CRUD citas + formulario con selects dinámicos + cambio de estado + PDFGenerator para comprobante + ConsultaCitaServlet con CAPTCHA visual (el ciudadano no tiene correo registrado, no es posible OTP)	RegistroVacunacionServlet.java + ConsultaPublicaServlet.java + PDFGenerator.java	Demo: programar cita → cambiar estado → consulta pública → descargar PDF
Semana	Módulos a completar	Referencia en el código de vacunación	Entregable de control
Semana 3	CSS propio (saludboyaca.css) + DashboardServlet con estadísticas por rol + Dockerfile + imagen en Docker Hub + despliegue en 2 hosting + README.md + video demo	styles.css del sistema de vacunación (solo como referencia, NO copiar) + Dockerfile del taller	Imagen en Docker Hub + 2 URLs activas + video demo de 3-5 min

11. Preguntas de Reflexión

 Responde en tu bitácora — mínimo 5 líneas por pregunta

Las respuestas de las secciones 11.1 y 11.2 se entregan al inicio del trabajo autónomo.

La sección 11.3 se responde al finalizar el proyecto SaludBoyacá.

11.1 Sobre el Código de Referencia y Arquitectura

- Estudiaste el LoginServlet del sistema de vacunación y luego creaste el LoginServlet de SaludBoyacá. ¿Cuántas líneas de código son iguales o muy similares? ¿Cuáles tuviste que cambiar completamente? ¿Qué te dice esto sobre el valor de estudiar código bien estructurado antes de escribir el propio?

RTA: Al comparar ambos servlets, calculo que aproximadamente un 60% del código es muy similar.

¿Qué cambié completamente? La lógica de validación post-login. En el sistema de vacunación se valida un CAPTCHA, mientras que en saludboyaca tuve que implementar la generación del código OTP y el servicio de envío de correos

Aprendizaje: Estudiar código bien estructura me dio el “esqueleto” funcional. No tuve que perder tiempo pensando en cómo manejar la petición HTTP, sino que pude concentrarme directamente en la lógica de negocio (seguridad por OTP), que es lo que hace único a mi proyecto

- El patrón switch/acción de PacienteServlet.java se repite casi idéntico en CitaServlet.java. ¿Por qué los buenos sistemas tienen este tipo de patrones repetibles? ¿Qué problema causaría si cada Servlet fuera completamente diferente en estructura?

RTA: El hecho de que PacienteServlet y CitaServlet compartan la misma estructura de switch(accion) es una de las mayores ventajas del proyecto

¿Por qué repetir patrones? Los buenos sistemas usan patrones repetibles porque reducen la carga cognitiva. Si yo, como desarrollador, entiendo cómo funciona el CRUD de pacientes, automáticamente sé cómo funciona el de citas, médicos o consultorios.

¿Qué pasaría si fueran diferentes? Si cada Servlet tuviera una estructura única, el mantenimiento sería una pesadilla. Cada error requeriría un análisis desde cero, aumentaría la probabilidad de bugs y haría que escalar el sistema fuera mucho más lento y costoso

- Compara el esquema de la BD de vacunación (5 tablas) con el de SaludBoyacá (7 tablas). ¿Qué decisiones de diseño tomaste diferentes y por qué? ¿Qué aprendiste del esquema de referencia que aplicaste directamente?

RTA: Aunque el esquema de vacunación fue mi punto de partida, las necesidades de SaludBoyacá me obligaron a expandir el diseño.

Decisiones diferentes:

Especialidades Médicas: Pasé de un esquema simple de “vacunadores” a uno de “médicos” vinculados a “especialidades”, lo que añadió tablas maestras adicionales

Estados de Cita: Implementé una tabla de estados (Pendiente, Cancelada, Completada) para rastrear el ciclo de vida de la cita, algo que en vacunación era más lineal.

¿Qué apliqué directamente? La gestión de Llaves Foráneas (FK) y la normalización. Del esquema de referencia aprendí la importancia de separar los datos del usuario (login) de sus datos como paciente (perfil), manteniendo la integridad referencial para que no queden "citas huérfanas" si se elimina un registro.

11.2 Sobre Internacionalización y OTP

- LocaleFilter.java del sistema de vacunación fue reutilizado casi sin cambios en SaludBoyacá. ¿Qué característica del diseño de esa clase la hace reutilizable? ¿Qué habría que cambiar si quisieras agregar un idioma nuevo (por ejemplo, portugués) a tu aplicación?

RTA: La característica principal que hace que esta clase sea tan "enchufable" es el desacoplamiento. El filtro no conoce el contenido de la aplicación; solo se encarga de interceptar la petición, buscar un parámetro (como ?lang=it) y configurar el Locale en la sesión del usuario.

¿Qué cambiaría para agregar portugués? 1. Crear un nuevo archivo messages_pt.properties con las traducciones. 2. En el header.jsp, añadir la opción visual (bandera de Brasil o Portugal) con el link ?lang=pt. 3. El filtro funcionaría automáticamente, ya que está diseñado para aceptar cualquier código ISO que reciba por parámetro.

- El CAPTCHA gráfico protege contra bots automatizados. El OTP por correo también lo hace, pero además verifica la identidad. ¿Por qué entonces el OTP reemplaza al CAPTCHA en el login y no se usan juntos? ¿En qué módulo de SaludBoyacá sigue siendo necesario el CAPTCHA y por qué no se puede reemplazar allí?

RTA: En el Login, el OTP reemplaza al CAPTCHA porque cumple una doble función: detiene a los bots (un bot no puede leer un correo electrónico externo fácilmente) y

valida que el usuario es quien dice ser. Usar ambos juntos sería fricción innecesaria para el usuario; entrar a una cuenta médica ya es un proceso serio, y poner demasiados obstáculos podría hacer que un paciente desista.

¿Dónde sigue siendo necesario el CAPTCHA? En la Consulta Pública de Citas.

¿Por qué no usar OTP ahí? Porque en la consulta pública no conocemos el correo del usuario hasta que él ingresa su documento. El CAPTCHA es la única barrera para evitar que un atacante haga "scraping" masivo (consultar miles de documentos por segundo para robar información de pacientes).

- El código OTP se genera con SecureRandom. ¿Por qué se usa SecureRandom en lugar de la clase Random normal de Java? Si guardas el OTP solo en sesión HTTP vs en la tabla otp_tokens de la BD, ¿cuál es más seguro y por qué? ¿Qué escenario de ataque resuelve guardarlo en la BD?

RTA: Para la generación del código, la elección de la clase es crítica:

SecureRandom vs. Random: Se usa SecureRandom porque es criptográficamente fuerte. La clase Random normal es predecible: si un atacante conoce la semilla (seed) y el tiempo del sistema, podría adivinar los siguientes códigos. SecureRandom genera números con una entropía mucho mayor, imposible de predecir.

Sesión HTTP vs. Base de Datos: Guardarlo en la Base de Datos es más seguro

¿Qué ataque resuelve? El escenario de secuestro de sesión (Session Hijacking) o ataques de fuerza bruta distribuidos. Si guardo el OTP en la BD con una marca de tiempo (created_at) y un estado (usado), puedo invalidar el código automáticamente después de 5 minutos o tras el primer intento fallido, algo que es más volátil y difícil de controlar estrictamente solo con sesiones de servidor

11.3 Sobre Docker, Despliegue y Decisiones de Producción

- El Dockerfile de SaludBoyacá usa Multi-Stage Build. Si no lo usaras y usaras una sola etapa con maven:3.8.6-openjdk-14, ¿cuánto pesaría la imagen? ¿Por qué ese tamaño importa en Back4App (256 MB de RAM) o en Koyeb (0.1 vCPU)?

RTA: Si no usara Multi-Stage Build y me quedara con la imagen de Maven 3.8.6 con OpenJDK 14, la imagen pesaría fácilmente entre 600 MB y 800 MB, ya que incluiría todas las herramientas de compilación, el código fuente y las dependencias descargadas. Al usar Multi-Stage, solo paso el archivo .war a una imagen de Tomcat, reduciendo el tamaño a unos 200-300 MB

Este tamaño es crítico para el rendimiento en hostings limitados como Back4App o Koyeb. En Back4App, con solo 256 MB de RAM, una imagen pesada consume mucha memoria solo al intentar descomprimirse e iniciar los procesos base, lo que causa errores de OutOfMemory antes de que la app cargue. En Koyeb, con 0.1 vCPU, una imagen grande tarda demasiado tiempo en procesar las capas durante el despliegue, haciendo que el inicio sea extremadamente lento y propenso a fallar por tiempos de espera (timeouts)

- Desplegaste SaludBoyacá en 2 hosting. Describe exactamente qué pasos seguiste, qué errores encontraste y cómo los resolviste. ¿Cuál fue el error más difícil de depurar en producción? ¿Cómo ves los logs de un contenedor en Render o Koyeb?

RTA:Desplegué SaludBoyacá en Render y Koyeb siguiendo estos pasos: Primero, subí el proyecto a GitHub asegurándome de que el Dockerfile estuviera en la raíz. En Render, conecté el repositorio y configuré las variables de entorno para la base de datos (DB_URL, DB_USER, DB_PASS). En Koyeb, seguí un proceso similar eligiendo el despliegue basado en Docker.

El error más difícil de depurar fue el de las variables de entorno en producción.

Localmente todo funcionaba, pero en el servidor la conexión a la base de datos fallaba porque la URL de la base de datos de Supabase que usé requería SSL, y no lo había configurado correctamente en el ConnectionFactory. Lo resolví añadiendo

"?useSSL=true" a la variable de entorno en el panel del hosting

Para ver los logs, en ambos casos es sencillo: en el panel lateral de Render hay una pestaña llamada Logs que muestra la consola de Tomcat en tiempo real. En Koyeb, dentro del servicio, hay una sección de Runtime Logs donde pude ver las excepciones de Java cuando fallaba la conexión

- Eres el desarrollador de SaludBoyacá y el Centro de Salud de Paipa te dice que van a tener 50 usuarios simultáneos. Evaluando los 5 hosting del taller, ¿cuál recomendarías para producción real? ¿Cuál sería tu segunda opción? Justifica con los datos de tu tabla comparativa (sección 8).

RTA: Recomendación para el Centro de Salud de Paipa Para un escenario de 20 usuarios simultáneos reales, mi recomendación definitiva para producción es Oracle Cloud.

Según los datos de mi tabla, es el único que ofrece hasta 6 GB de RAM en su capa gratuita. Con 50 usuarios, SaludBoyacá necesitará gestionar muchas sesiones HTTP y procesos de envío de correos OTP al mismo tiempo, y los 512 MB de Render o Koyeb se quedarían cortos muy rápido, causando caídas del servicio

Mi segunda opción sería Koyeb. Aunque tiene menos RAM que Oracle, su ventaja es que no se duerme. Esto es vital para un centro de salud; no podemos permitir que un médico o un paciente tenga que esperar 30 segundos a que el servidor de Render despierte cuando intenten agendar una cita urgente. Además, su tiempo de deploy es el más rápido de la comparativa, lo que permitiría aplicar parches de seguridad de forma casi instantánea

12. Criterios de Evaluación

La evaluación tiene dos componentes: las sesiones de laboratorio (40%) y el proyecto final SaludBoyacá (60%).

12.1 Sesiones de Laboratorio — 40%

#	Criterio	Puntos	Instrumento
1	Sesión 1: Bitácora completa con análisis de los 8 componentes del sistema de vacunación	10 %	Revisión bitácora
2	Sesión 2: Sistema de vacunación con OTP funcional (sin CAPTCHA en login) + imagen en Docker Hub	20 %	Demostración URL

3	Sesión 2: Desplegado en Render y Koyeb con flujo OTP funcionando en producción	10 %	Revisión URLs
---	--	------	---------------

12.2 Proyecto Final SaludBoyacá — 60% (ver rúbrica detallada en sección 9.10)

#	Criterio resumido	Puntos	Instrumento
1	Script SQL con 7 tablas + datos de prueba reales de Paipa	8 %	Revisión SQL
2	Arquitectura correcta — cero SQL en Servlets, patrón DAO	10 %	Revisión código
3	i18n en 3 idiomas (es, en, it) — cero textos hardcodeados en JSP	12 %	Demostración
4	Flujo OTP completo: login sin CAPTCHA → OTP por correo → dashboard. CAPTCHA solo en consulta pública (sin cuenta)	10 %	Demostración
5	CRUD de citas + cambio de estado + PDF comprobante	12 %	Demostración
6	Consulta pública: CAPTCHA visual (no hay OTP sin cuenta) + i18n + descarga PDF	8 %	Demostración
#	Criterio resumido	Puntos	Instrumento
7	CSS propio saludboyaca.css con paleta definida — no copia styles.css	8 %	Revisión CSS
8	Dockerfile + imagen Docker Hub + 2 hosting activos + video demo	12 %	Revisión URLs

Condiciones de NO aprobación automática

- SQL hardcodeado dentro de Servlets.
- PreparedStatement sustituido por concatenación de Strings (SQL Injection).
- Aplicación que no compila o no despliega en Tomcat local.
- i18n implementado con textos hardcodeados en JSP (no usa <fmt:message>).

- CAPTCHA en el formulario de login (debe ser reemplazado por OTP).
- CSS copiado directamente de styles.css del sistema de vacunación.
- Código del sistema de vacunación copiado sin adaptación (mismo dominio, mismas entidades).

13. Errores Frecuentes y Soluciones

Error / Síntoma	Causa y Solución
Los textos no se traducen aunque cambies ?lang=en	El archivo messages_en.properties no está en src/main/resources/ o el nombre está mal escrito. Verifica: ls src/main/resources/ desde la terminal.
Tildes y ñ aparecen como signos de interrogación	El .properties no está en UTF-8. NetBeans: clic derecho → Properties → Encoding = UTF-8. Alternativa: usar \u00f3 en lugar de 'ó'.
El OTP nunca llega al correo	Gmail bloqueó el envío. Causa 1: usaste la contraseña normal en vez de App Password. Causa 2: no tienes 2FA activado en Gmail. Causa 3: firewall corporativo bloqueó SMTP 587.
AuthException: 535 Authentication failed	El EMAIL_PASS tiene espacios en la variable de entorno. Enciérralo en comillas: EMAIL_PASS='xxxx xxxx xxxx xxxx'.
Dashboard accesible sin ingresar OTP	Olvidaste agregar && Boolean.TRUE.equals(session.getAttribute('otpVerificado')) en AuthFilter.
NullPointerException en LoginServlet tras refactorizar	Quedó alguna línea que lee session.getAttribute('captchaText') o llama a CaptchaGenerator. Revisa que eliminaste TODAS las referencias al CAPTCHA en LoginServlet.doGet() y doPost().
El formulario login.jsp sigue mostrando el campo CAPTCHA	No eliminaste el bloque div.captcha-container del JSP. Borra todo desde <div class='captcha-container'> hasta su </div>, incluyendo el img y el input name='captcha'.
docker build falla: 'mvn: not found'	La imagen eclipse-temurin no tiene Maven. Usa maven:3.8.6openjdk-14 en la etapa de build del Dockerfile.
Error / Síntoma	Causa y Solución
Tomcat no despliega el WAR en Docker	El WAR no está en /usr/local/tomcat/webapps/ROOT.war. Verifica con: docker exec -it id ls /usr/local/tomcat/webapps/

Cannot connect to database en el hosting	La BD en Railway no es accesible. Verifica: DB_URL tiene el host correcto, el firewall de Railway permite conexiones externas y las credenciales son correctas.
Back4App: OutOfMemoryError al iniciar	256 MB insuficientes. Agrega JAVA_TOOL_OPTIONS=-Xms64m -Xmx200m como variable de entorno en Back4App.
El idioma vuelve a español al reiniciar Tomcat	Comportamiento correcto — la sesión HTTP se destruye. En producción usar CookieLocaleResolver (ver reto avanzado de la sesión 1).

SENA · Servicio Nacional de Aprendizaje

Centro Industrial de Mantenimiento y Manufactura – CIMM | Regional Boyacá Tecnólogo
en Análisis y Desarrollo de Software – ADSO | 2026